AGENT DATA PROTOCOL：用于多样且高效的大语言模型智能体微调的数据集统一协议

Yueqi Song[1], **Ketan Ramaneti**[1], **Zaid Sheikh**[1], **Ziru Chen**[2], **Boyu Gou**[2], **Tianbao Xie**[3], **Yiheng Xu**[3], **Danyang Zhang**[3], **Apurva Gandhi**[1], **Fan Yang**[5], **Joseph Liu**[1], **Tianyue Ou**[1], **Zhihao Yuan**[1], **Frank Xu**[1], **Shuyan Zhou**[4], **Xingyao Wang**[6], **Xiang Yue**[1], **Tao Yu**[3], **Huan Sun**[2], **Yu Su**[2], **Graham Neubig**[1,6]

[1]Carnegie Mellon University, [2]The Ohio State University, [3]University of Hong Kong, [4]Duke University, [5]Fujitsu Research, [6]All Hands AI
{yueqis,gneubig}@cs.cmu.edu

https://agentdataprotocol.com

ABSTRACT

面向 AI 智能体的大规模监督微调公开研究结果仍然相对稀少，因为收集智能体训练数据存在独特挑战。本文认为，瓶颈并不在于缺少底层数据源，而在于大量数据分散在异构的格式、工具和接口之中。为此，我们提出 Agent Data Protocol（ADP），一种轻量级表示语言，可作为不同格式智能体数据集与下游统一智能体训练流水线之间的“中间语”。ADP 的设计既具有足够的表达能力，可覆盖 API/工具使用、浏览、编码、软件工程以及一般智能体工作流等多种任务，同时又保持足够简洁，无需针对每个数据集单独做工程化处理即可解析和训练。在实验中，我们将现有的 13 个智能体训练数据集统一为 ADP 格式，并将标准化后的 ADP 数据转换为适用于多个智能体框架的训练格式。我们在统一后的数据上进行了监督微调，结果表明，相比对应的基础模型，平均性能提升约 ∼20%，并且在标准编码、浏览、工具使用与研究类基准上，在无需领域特定调优的前提下取得了当前最优或接近当前最优的表现。我们公开发布全部代码和数据，希望 ADP 能够降低标准化、可扩展且可复现的智能体训练门槛。

1 引言

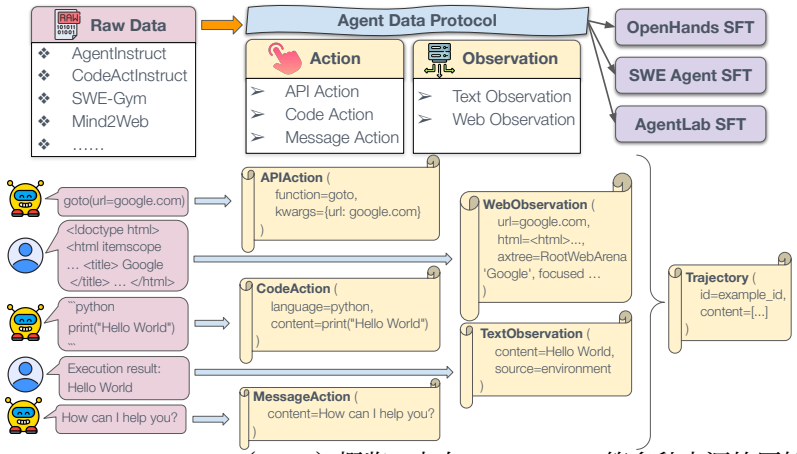

Figure 1: Agent Data Protocol（ADP）概览。来自 SWE-Gym 等多种来源的原始数据会被转换为标准化的 ADP 格式。ADP 将数据统一表示为 Trajectory 对象，其中包含两个核心组成部分：Actions（API action、code action、message action）与 Observations（text observation、web observation），从而能够无缝接入多种智能体 SFT 格式。图中的转换示例展示了如何将异构原始数据规范化，以训练智能体模型。

大语言模型（LLM）的预训练得益于海量且易于获取的互联网规模数据。相比之下，后训练面临的挑战要困难得多：高质量、面向特定任务的数据必须经过精心构建。尽管在相对简单的场景中，人们已经提出了许多创造性的数据收集策略，例如代码生成 (Nijkamp et al., 2023)、问答 (Rajpurkar et al., 2016) 和情感分析 (Maas et al., 2011) 等单轮用户交互任务，但许多真实世界任务远比这些复杂。

其中一个特别困难的场景是智能体应用：模型需要连续采取动作，并与外部世界进行迭代交互。为这类场景构建数据集，需要记录并组织智能体行为轨迹，这比收集静态输入输出对要困难得多。

尽管存在这些困难，越来越多的工作仍在探索构建智能体数据集的不同方法。这些工作的方法论各不相同，包括人工整理 (Rawles et al., 2023; Xu et al., 2024a)、合成数据生成 (Ou et al., 2024; Zheng et al., 2024a) 以及记录智能体 rollout (Pan et al., 2025; Yang et al., 2025b)。由此产生的数据集覆盖了广泛的任务类型，包括网页导航 (Deng et al., 2023; Lù et al., 2024)、软件开发 (Yang et al., 2025b; Pan et al., 2025)、视觉界面控制 (Rawles et al., 2023; Kapoor et al., 2024) 和通用工具使用 (Zeng et al., 2023; Liu et al., 2024a)（这些数据集的概览见 § 2.1）。

然而，尽管这类数据已经存在，面向智能体的大规模监督微调（SFT）在学术研究中仍然相当罕见。少数有代表性的项目，如 Zeng et al. (2023) 和 Mitra et al. (2024)，已经展示了这一路线的潜力，但它们仍是例外而非常态。为什么这还没有成为标准做法？ 我们认为，问题不在于缺少数据，而在于缺少标准化。现有数据集高度碎片化，格式与表示方式不一致，导致它们难以被有效地组合、共享和利用，因此长期处于未被充分使用的状态。

为弥补这一缺口，我们提出 Agent Data Protocol（ADP），一种面向智能体数据的标准化且具有表达能力的表示语言。通过将异构数据集转换为 ADP，可以方便地为多种下游训练流水线生成大规模且多样化的数据（Figure 1）。在技术实现上，ADP 采用 Pydantic[1] 模式来表示与常见智能体使用场景相关的动作和观察，例如通信、浏览、编码以及各类工具调用，并辅以严格的自动化校验以维持较高的数据质量。

作为展示 ADP 实际效用的第一步，我们实现了从 13 个现有数据集到 ADP 的转换器，以及从 ADP 到 3 种不同智能体架构的转换器，以展示其通用性。基于这些工作，我们构建并发布了目前公开可获得的最大规模智能体训练数据集，其中包含 1.3M 条训练轨迹，命名为 ADP Dataset V1。

实验表明，使用 ADP 训练智能体能够在多个领域带来显著性能提升，包括编码（SWE-Bench Verified）、网页浏览（WebArena）、研究任务（GAIA）以及智能体工具使用（AgentBench），详见 § 6。值得注意的是，这些结果相较基础模型平均提升了 20%，并且与同规模模型的其他当前最优结果相比具有竞争力，甚至更优。我们还发现显著的跨任务迁移收益：基于 ADP 数据训练，明显优于仅在单一数据集上训练。除性能之外，ADP 还支持系统性的跨数据集分析，从而揭示公开数据中的趋势与改进空间。

最后，我们将全部代码和数据集开源，以促进社区采用并鼓励贡献新的数据集。我们相信，ADP 通过提供使大规模监督式智能体训练变得切实可行且可扩展所需的标准化，将推动智能体模型微调进入下一波发展阶段。

2 相关工作

高效的基于 LLM 的智能体，其发展关键依赖于高质量训练数据，而这些数据需要能够刻画多步推理、工具使用和环境交互的复杂性 (Yao et al., 2022b; Schick et al., 2023; Deng et al., 2023; Masterman et al., 2024)。本节回顾现有的智能体数据收集方法，以及推动 ADP 出现的核心挑战。

2.1 智能体数据收集方法

现有方法涵盖人工构造（由人类专家逐步编写所需智能体行为示范）(Nakano et al., 2021; Yao et al., 2022a)、合成生成（利用现有 LLM 通过提示或结构化生成构造智能体轨迹）(Luo et al., 2023; Xu et al., 2024b)，以及记录智能体 rollout（捕获现有智能体系统在执行任务过程中的轨迹）(Wang et al., 2024a; Pan et al., 2025) 等，由此产生了丰富的智能体训练数据，其中具有代表性的一组列于 Table 1。

我们还将每个数据集归入一个粗粒度任务类别。

[1] `https://pydantic.dev/`

Table 1: 现有智能体训练数据集概览。C=Coding，S=Software Engineering，T=API/Tool Use，W=Web Browsing。

数据集	类别	数量	来源	说明
AgentInstruct (Zeng et al., 2023)	C T W	1.9K	合成	浏览、数据库、操作系统等任务的混合
Code-Feedback (Zheng et al., 2024a)	C	66.4K	人工	带运行时反馈闭环的代码生成
CodeActInstruct (Wang et al., 2024b)	C	7.1K	合成	带执行的代码生成与工具使用
Go-Browse(Gandhi & Neubig, 2025)	W	9.5K	rollout	结构化探索式网页 rollout
Mind2Web (Deng et al., 2023)	W	1.0K	人工	真实网站上的人工网页操作示范
Nebius SWE Trajectories (Golubev et al., 2024)	S	13.4K	rollout	Nebius 基于纯开源权重模型生成的 SWE-agent 轨迹
NNetNav-live (Murty et al., 2024)	W	5.0K	rollout	事后标注的真实网页探索
NNetNav-wa (Murty et al., 2024)	W	4.2K	rollout	事后标注的 WebArena 探索
openhands-feedback (All Hands AI, 2024)	C T W	0.2K	rollout	带人工反馈的 OpenHands 智能体轨迹记录
Orca Agentinstruct (Mitra et al., 2024)	T	1046.1K	合成	大规模合成工具使用指令数据
SWE-Gym (Pan et al., 2025)	S	0.5K	rollout	解决真实 GitHub 仓库任务的智能体轨迹
SWE-smith (Yang et al., 2025b)	S	5.0K	合成	智能体在合成 bug 修复任务上的轨迹
Synatra (Ou et al., 2024)	W	99.9K	合成	合成生成的教程式网页示范

- **Coding**：通常包括基础编程任务，例如命令行代码生成、算法实现、代码补全、代码翻译和代码修复等。
- **Software Engineering**：通常由仓库级软件工程任务组成，例如 bug 修复、功能实现、代码重构和依赖管理等。
- **API/Tool Use**：通常要求智能体有效使用外部 API/工具来解决任务。常见工具包括文件操作、数据库查询以及定制 API 等。
- **Web Browsing**：通常包括网页导航、在线购物和社交媒体交互等任务，需要智能体理解图形界面。

2.2 挑战与局限

尽管现有智能体训练数据集已经相当丰富，但仍有若干基础性挑战阻碍了这些资源被有效地大规模利用：

- **数据构建的复杂性**：构建高质量智能体训练数据需要大量资源和专业知识 (Paullada et al., 2021; Bhardwaj et al., 2024; Zha et al., 2025)。人工整理成本高且依赖领域知识；合成生成面临数据质量验证难题；记录智能体 rollout 则从根本上受限于现有基线智能体的能力，从而限制了轨迹的多样性和复杂性。尽管近期已有工作扩大了轨迹收集规模 (Song et al., 2024; Mitra et al., 2024)，但如何在不同构建方式之间平衡质量、多样性与规模，仍是根本挑战。
- **数据集格式的异构性**：现有智能体训练数据集各自采用不同的表示格式、动作空间和观察结构 (Ning et al., 2025; Luo et al., 2025)。例如，一些网页数据集使用 HTML，而另一些使用无障碍树结构 (de Chezelles et al., 2025)。现有工作已经注意到并开始尝试解决数据标准化问题 (Zhang et al., 2024; Chen et al., 2024; Mohammadi et al., 2025; Xi et al., 2025; Zhang et al., 2025)，但大多聚焦于任务特定或智能体特定的统一，而不是社区范围内的数据表示标准化，因此难以与其他数据集或智能体实现即插即用。要联合使用多个数据集，仍需投入大量工程工作，阻碍了不同数据源之间的整合。
- **分析与比较的困难**：现有数据集结构的多样性，也使得跨不同数据源进行系统性比较或定量分析变得困难 (Putrama & Martinek, 2024)，限制了研究者理解不同数据集相对效用、覆盖范围和质量的能力，也阻碍了基于数据驱动的选择与改进。

3 Agent Data Protocol

为克服上述挑战与局限，并更好地利用现有数据资源，我们提出 Agent Data Protocol（ADP）。ADP 建立了一套统一模式，用来弥合现有异构智能体训练数据集与大规模监督式智能体微调之间的鸿沟。

3.1 设计原则

我们围绕以下核心原则来设计 ADP：

- **简洁性**：ADP 保持简单直观的结构。它通过提供一个直接明了的框架，消除了针对每个数据集单独工程适配的需要，从而直接应对了数据构建复杂性这一挑战，使研究者无需投入大量适配工作，也能利用大规模智能体数据。
- **标准化**：ADP 的目标是提供一种统一表示，将现有各种不同格式的智能体训练数据集标准化到同一种格式中，从而应对数据集格式异构性的挑战。
- **表达能力**：ADP 被设计为能够准确表达复杂的智能体轨迹，而不丢失关键信息。它也直接回应了分析与比较困难这一挑战，因为 ADP 具有足够强的表达能力，可以覆盖不同领域中种类繁多的现有智能体数据集，使研究者能够在统一条件与上下文中考察这些多样化数据。

通过解决智能体数据利用中的这些基础性难题，ADP 旨在推动智能体训练的发展，使大规模智能体 SFT 对更广泛的研究社区更加可及。

3.2 架构

ADP 模式以 Pydantic schema 实现，设计上既简洁又具备表达力。每条经 ADP 标准化的智能体轨迹都表示为一个 `Trajectory` 对象。

Trajectory 包含三部分：(1) `id`：轨迹标识；(2) `content`：由动作与观察交替组成的序列，用于表示智能体与用户/环境之间的交互；(3) `details`：灵活的元数据字典，用于存储数据集特定信息（例如数据集来源 URL）。

Action 表示智能体的决策与行为。我们将动作分为三类：

- **API Actions**：带结构化参数与输出的函数调用，用于刻画工具使用。每个 API action 包含：(1) `function`：工具调用名称；(2) `kwargs`：函数参数字典；(3) `description`：可选的动作推理或说明。例如，在 ADP 中，一个网页导航调用 `goto(url=https://www.google.com)` 可表示为 `APIAction(function=goto, kwargs=url:https://www.google.com)`。
- **Code Actions**：跨编程语言的代码生成与执行。每个 code action 指定：(1) `language`：编程语言（例如 python）；(2) `content`：待执行代码；(3) `description`：可选的动作推理或说明。例如，一个 python 代码块 ```` ```python print("Hello World")``` ```` 在 ADP 中可表示为 `CodeAction(language=python,content=print("Hello World"))`。
- **Message Actions**：智能体与用户之间的自然语言通信，每个 action 均包含 `content` 字段，用于记录智能体的解释、澄清与回复。例如，`MessageAction(content=How can I help you?)`。

Observation 表示智能体从环境中获得的感知，分为两类：

- **Text Observations**：捕获来自不同来源的文本信息，包括用户指令和环境反馈。每个 text observation 包含：(1) `source`：观察来源（“user” 或 “environment”）；(2) `content`：观察到的文本内容。例如，一段 python 执行输出 `Execution result: Hello World` 会被转换为 ADP 格式 `TextObservation(content=Hello World, source=environment)`。
- **Web Observations**：表示网页的状态与内容。每个 observation 包含：(1) `html`：原始 HTML 内容；(2) `axtree`：网页的无障碍树；(3) `url`：当前页面 URL；(4) `viewport_size`：浏览器视口尺寸；(5) `image_observation`：可选的截图数据。Web observations 使 ADP 能够支持复杂的浏览场景。

ADP 的核心洞见在于：尽管智能体数据集在表面形式上多种多样，但大多数智能体交互都可以分解为智能体采取的一系列 *actions* 以及从环境接收到的一系列 *observations*。通过标准化这些基础组成部分，ADP 在保留原始数据丰富语义的同时，直接回应了 § 2.2 中提出的每项挑战。这种统一表示使研究者能够组合此前彼此不兼容的数据集，从而推动跨多领域的大规模训练。

3.3 转换流水线

如 Figure 1 所示，我们基于 ADP 实现了一个三阶段转换流水线，可将异构数据集转换为可直接用于训练的智能体格式。

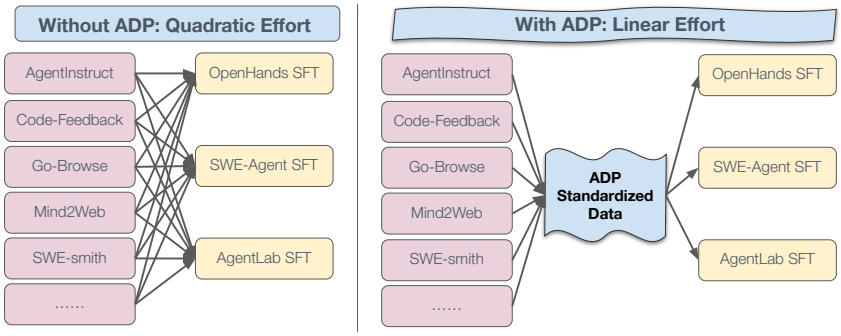

Figure 2: **ADP 将多对多转换收敛为轮毂-辐条式流水线。左：**没有 ADP 时，D 个数据集中的每一个都需要针对 A 种智能体格式分别编写一个定制的 Raw→SFT 转换器（工作量为二次的 $O(D \times A)$），导致代码与人力投入重复。**右：**有了 ADP 之后，每个数据集只需转换一次（Raw→ADP），每个智能体也只需要一个转换器（ADP→SFT），总工作量变为线性的 $O(D+A)$。新的数据集或智能体可以立即接入 ADP 的其余部分。

1. **Raw to Standardized**：这一阶段将原始数据集格式统一到 ADP 标准化 schema 中。每个数据集先以原始格式提取，再通过将该数据集特定的 actions 和 observations 映射到 ADP 的标准动作与观察空间，转换为 ADP schema。例如，带 HTML 表示的网页浏览任务会被转换为 `APIAction` 与 `WebObservation` 配对，而带执行输出的编码任务则映射为 `CodeAction` 与 `TextObservation` 配对。
2. **Standardized to SFT**：这一阶段将 ADP 标准化轨迹转换为适合训练语言模型的监督微调（SFT）格式。不同的智能体框架使用不同的动作空间、观察格式等。例如，OpenHands 采用带网页浏览能力的 IPython 执行环境，SWE-Agent 使用结构化 bash 命令与文件操作，而 AgentLab 则聚焦于基于 DOM 的网页交互。我们认识到，高效的智能体训练并不只是训练一个通用动作模型，而是需要适配每个框架特有的脚手架与交互格式。对每个 agent harness，转换过程都使用一个特定脚本，根据该框架的设计，将各类 action 和 observation 翻译到目标智能体的动作与观察空间中。这一阶段还负责上下文管理、系统提示设定，以及将对话整理成适合 SFT 的指令-响应对，以适配具体的智能体架构。
3. **Quality Assurance**：这一阶段通过自动化校验，确保数据在智能体格式、工具使用和对话结构层面的正确性与一致性。示例性的质量检查包括：验证工具调用格式、确保大多数[2]工具调用都配有英文 thought，以及检查对话是否正确结束等。

3.4 ADP 对智能体训练研究的实际影响

双向流水线（Raw→ADP 与 ADP→SFT）清晰地分离了职责，并消除了重复工程工作（Figure 2）。在实践中：

- **数据集转换（每个数据集一次）**。贡献者只需将每个**原始**数据集转换到 *ADP* schema **一次**。此后，该数据集就成为任何 agent harness 都可使用的标准化资源。
- **智能体特定转换（每个智能体一次）**。每个智能体只需维护一份 *ADP→SFT* 脚本；不再需要针对每个数据集单独做工程适配。新增数据集时，**无需**修改智能体侧脚本。

ADP 将转换成本在整个社区范围内摊薄，加速了新数据集的采用，并保证一份 ADP→SFT 脚本就能让某个智能体框架立即使用全部 ADP 标准化数据。没有 ADP 时，研究者必须为**每一个**数据集-智能体配对单独编写 *Raw→SFT* 转换器，不仅跨团队重复劳动，也会让大规模数据整合变得脆弱而缓慢。更多讨论见 § 6.3。

4 跨数据集分析

Table 2 展示了对 13 个经 ADP 标准化数据集的分析结果，揭示了不同数据集之间显著的多样性。

Table 2: 数据集统计与轨迹分析。A=APIAction, C=CodeAction, M=MessageAction。

数据集	平均轮数	动作占比 (A/C/M)	带函数 Thought 占比
AgentInstruct	8.2	64/10/26	100.0
Code-Feedback	4.0	0/58/42	82.8
CodeActInstruct	4.0	0/65/35	98.6
Go-Browse	3.9	70/0/30	100.0
Mind2Web	9.7	90/0/10	0.0
Nebius SWE-Agent	16.2	67/27/6	100.0
NNetNav-live	8.2	80/0/20	99.9
NNetNav-wa	10.1	89/0/11	99.9
OpenHands	18.3	11/73/16	91.7
Orca AgentInstruct	1.3	0/15/85	84.0
SWE-Gym	19.7	61/25/14	42.0

[2]我们将该阈值设为 80%，但可以按需要调整。

轨迹长度。不同数据集的轨迹轮数差异很大，从 1 轮到 26.8 轮不等，平均为 10.1 轮。SWE 数据集通常更长，这反映了仓库级编程任务本身的复杂性。

动作分布。经 ADP 标准化后，动作分布呈现出明显的领域特征。Web 数据集（如 Mind2Web）高度偏向 API actions，几乎不进行代码执行，这反映了它们对界面交互的关注。相反，编码数据集（如 Code-ActInstruct）大量使用 code actions 而几乎不使用 API，强调直接的编程活动。SWE 数据集（如 SWE-smith）则呈现混合模式，既依赖文件写入等 API actions，也使用 code actions 进行代码生成。

函数推理分析。一个显著发现是函数 thought 的覆盖率很高（大多数数据集达到 $\geq 90\%$），这表明这些训练数据集通常会为动作提供解释。这对于可解释性以及训练具备推理能力的智能体尤其重要。更重要的是，高推理覆盖率出现在所有任务类别中，说明函数 thought 更像是记录完善数据集的一种普遍特征，而非某个特定领域的专有行为。

5 实验设置

5.1 训练设置

为了评估 ADP 在多样化数据源上的训练有效性，我们使用了一组全面的 13 个智能体训练数据集，覆盖编码、SWE、API/工具使用和浏览等任务类型，如 Table 1 所示。这些数据集体现了 ADP 所要解决的一系列异构性挑战，包括不同的数据创建方式（合成生成、人工整理、智能体 rollout）、不同的复杂度（从简单任务到复杂的多步工作流）以及不同的环境（命令行界面、网页 GUI、Jupyter Notebook、API 调用）。

所选数据集总计包含超过 1.3M 个样本，既包括 Mind2Web 这类较小数据集，也包括 Orca AgentInstruct 这类大规模数据集。为了确保不同领域之间的表示平衡，并避免某个大型数据集主导训练过程，我们对较大的数据集进行子采样，而对较小的数据集则完整使用。关于数据采样和混合权重的完整细节见 Appendix C。

我们使用 Qwen2.5-Coder-Instruct 模型家族 (Qwen Team, 2024; Hui et al., 2024) 作为基础模型，并结合 3 个智能体框架，在多个基准上进行综合评估。所有模型都使用同一套来自 LLaMA-Factory (Zheng et al., 2024b) 的 SFT 流水线进行微调。这些实验聚焦于各框架擅长的特定领域，以展示其针对性的有效性。每个智能体都具有独特的架构、工具接口和交互环境。这种多样性使我们能够验证：经 ADP 标准化的数据可以方便地转换为不同的智能体格式，从而证明该协议在多种智能体实现中的实用价值。

OpenHands (Wang et al., 2025) 是一个用于构建通用 AI 智能体的开放平台，这类智能体以类似软件开发者的方式工作：写代码、使用命令行并浏览网页。它提供沙箱执行环境、工具协同与基准评测能力。

AgentLab (Drouin et al., 2024; de Chezelles et al., 2025) 是一个开源框架，用于在多样化任务上开发、测试和评测网页智能体，强调可扩展性与可复现性。它支持 WebArena 和 WorkArena 等一系列评测基准。

SWE-Agent (Yang et al., 2024) 引入了一个定制的 Agent-Computer Interface（ACI），使语言模型智能体能够通过浏览代码库、编辑并运行代码、查看文件和执行测试等方式，自主完成软件工程任务。

5.2 评测基准

我们在 4 个跨不同领域的基准上评测这些智能体（具体选择基于评测代码的可用性以及各智能体的专长方向）。这一全面评估展示了 ADP 在保留多样任务关键语义信息方面的表达能力。

SWE-Bench (Jimenez et al., 2024) 用于评估智能体在真实世界软件工程任务上的表现。给定一个 GitHub 代码库和一份 bug 报告，智能体需要生成能够通过现有单元测试的补丁。我们使用 SWE-Bench Verified 子集进行评测 (Chowdhury et al., 2024)。

WebArena (Zhou et al., 2024) 提供了一个逼真的自托管网页环境，其中包含电商、论坛、地图导航等领域的完整可用网站，要求智能体理解高层自然语言指令并执行具体网页交互。

AgentBench (Liu et al., 2024b) 在操作系统、数据库和网页浏览等不同环境下评估智能体，强调多轮推理、决策能力以及跨领域适应性。

GAIA (Mialon et al., 2023) 是一个面向通用 AI 助手的基准，包含人工标注任务，这些任务结合了推理、工具使用和多步问题求解，并且经常带有多模态输入。任务难度会随步骤数和所需工具不同而变化。

6 实验结果

Table 3: SOTA 与我们最佳 7–8B ADP 训练智能体在各基准上的结果对比。阴影行是我们的 ADP 调优模型，其他行来自已有工作。

智能体	模型	训练数据	准确率
SWE-Bench (Verified) (Jimenez et al., 2024; Chowdhury et al., 2024)			
SWE-Agent (Yang et al., 2024)	`Qwen-2.5-7B-Coder-Instruct`	–	0.4%
	`Qwen-2.5-7B-Coder-Instruct`	SWE-smith (Yang et al., 2025b)	15.2% (+14.8%)
	`Claude 3 Opus` (Anthropic Team)	–	15.8%
	`Qwen-2.5-7B-Coder-Instruct`	ADP Data	20.2% (+19.8%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-7B-Coder-Instruct`	–	2.8%
	`Qwen-2.5-7B-Coder-Instruct`	SWE-Gym (Pan et al., 2025)	10.6% (+7.8%)
	`Qwen-2.5-7B-Coder-Instruct`	ADP Data	20.4% (+17.6%)
WebArena (Zhou et al., 2024)			
BrowserGym (de Chezelles et al., 2025)	`Llama-3.1-8B`	–	1.0%
	`Qwen-2.5-7B-Instruct`	–	8.3%
	`Llama-3.1-8B`	NNetNav (Murty et al., 2024)	16.3% (+15.3%)
	`Qwen-2.5-7B-Instruct`	Go-Browse (Gandhi & Neubig, 2025)	21.7% (+13.4%)
AgentLab (Drouin et al., 2024) (de Chezelles et al., 2025)	`Qwen-2.5-7B-Coder-Instruct`	–	4.5%
	`Qwen-2.5-7B-Coder-Instruct`	ADP Data	21.0% (+16.5%)
AgentBench OS (Liu et al., 2024b)			
AgentLM (Liu et al., 2024b)	`Llama-2-chat-7B`	–	8.3%
	`Llama-2-chat-7B`	AgentInstruct (Zeng et al., 2023)	17.4% (+9.1%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-7B-Coder-Instruct`	–	3.5%
	`Qwen-2.5-7B-Coder-Instruct`	ADP Data	27.1% (+23.6%)
GAIA (Mialon et al., 2023)			
OWL Agent (Hu et al., 2025)	`Qwen-2.5-7B-Instruct`	–	4.8%
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-7B-Instruct`	–	7.3%
	`Qwen-2.5-7B-Instruct`	ADP Data	9.1% (+1.8%)

Table 4: SOTA 与我们最佳 13–14B ADP 训练智能体在各基准上的结果对比。阴影行是我们的 ADP 调优模型，其他行来自已有工作。

智能体	模型	训练数据	准确率
SWE-Bench (Verified) (Jimenez et al., 2024; Chowdhury et al., 2024)			
SWE-Agent (Yang et al., 2024)	`Qwen-2.5-14B-Coder-Instruct`	–	2.0%
	`Claude 3.5 Sonnet`(Anthropic Team)	–	33.6%
	`Qwen-2.5-14B-Coder-Instruct`	ADP Data	34.4% (+32.4%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-14B-Coder-Instruct`	–	5.8%
	`Qwen-2.5-14B-Coder-Instruct`	SWE-Gym (Pan et al., 2025)	16.4% (+10.6%)
	`Qwen-2.5-14B-Coder-Instruct`	ADP Data	30.6% (+24.8%)
WebArena (Zhou et al., 2024)			
AgentLab (Drouin et al., 2024) (de Chezelles et al., 2025)	`Qwen-2.5-14B-Coder-Instruct`	–	5.5%
	`Qwen-2.5-14B-Coder-Instruct`	ADP Data	22.2% (+16.7%)
AgentBench OS (Liu et al., 2024b)			
AgentLM (Liu et al., 2024b)	`Llama-2-chat-13B`	–	9.0%
	`Llama-2-chat-13B`	AgentInstruct (Zeng et al., 2023)	18.1% (+9.1%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-14B-Coder-Instruct`	–	2.8%
	`Qwen-2.5-14B-Coder-Instruct`	ADP Data	20.8% (+18.0%)

Table 5: SOTA 与我们最佳 32B ADP 训练智能体在各基准上的结果对比。阴影行是我们的 ADP 调优模型，其他行来自已有工作。

智能体	模型	训练数据	准确率
SWE-Bench (Verified) (Jimenez et al., 2024; Chowdhury et al., 2024)			
SWE-Agent (Yang et al., 2024)	`Qwen-2.5-32B-Coder-Instruct`	–	2.2%
	`Qwen-2.5-32B-Coder-Instruct`	SWE-smith (Yang et al., 2025b)	40.2% (+38.0%)
	`Qwen-2.5-32B-Coder-Instruct`	ADP Data	40.3% (+38.1%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-32B-Coder-Instruct`	–	10.6%
	`Qwen-2.5-32B-Coder-Instruct`	SWE-Gym (Pan et al., 2025)	20.6% (+10.0%)
	`Qwen-2.5-32B-Coder-Instruct`	ADP Data	36.8% (+26.2%)
WebArena (Zhou et al., 2024)			
AgentLab (Drouin et al., 2024) (de Chezelles et al., 2025)	`Qwen-2.5-32B-Coder-Instruct`	–	10.9%
	`Qwen-2.5-32B-Coder-Instruct`	ADP Data	22.9% (+12.0%)
AgentBench OS (Liu et al., 2024b)			
AgentLM (Liu et al., 2024b)	`Llama-2-chat-70B`	–	9.0%
	`Llama-2-chat-70B`	AgentInstruct (Zeng et al., 2023)	21.5% (+12.5%)
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-32B-Coder-Instruct`	–	27.8%
	`Qwen-2.5-32B-Coder-Instruct`	ADP Data	34.7% (+6.9%)

6.1 ADP 数据在多样任务上训练出高效智能体

ADP 微调在不同模型、基准和 agent harness 上都能稳定提升性能。如 Table 3、Table 4 和 Table 5 所示，在标准化 ADP 数据上训练，能够让 7B、14B 和 32B 模型在多个流行评测基准上获得显著收益。在 *SWE-Bench (Verified)* 上，ADP 训练带来了非常显著的提升：对于 `Qwen-2.5-7B-Coder-Instruct`，在 SWE-Agent 上从 0.4% 提升到 20.2%（+19.8%），在 OpenHands 上从 2.8% 提升到 20.4%（+17.6%）。在 14B 规模下，`Qwen-2.5-14B-Coder-Instruct` 在 SWE-Agent 上达到 34.4%（+32.4%），在 OpenHands 上达到 30.6%（+24.8%）。32B 模型在 SWE-Agent 上达到 40.3%（+38.1%），在 OpenHands 上达到 36.8%（+26.2%），与 SWE-Agent 上 Claude 3.5 Sonnet 的 33.6% 表现持平甚至更优。在 *WebArena* 上，ADP 训练在不同模型规模下也表现出稳定增益: 7B 达到 21.0%（+16.5%），14B 达到 22.2%（+16.7%），32B 达到 22.9%（+12.0%）。在 *AgentBench OS* 上，提升同样明显：7B 模型从 3.5% 提升到 27.1%（+23.6%），14B 模型从 2.8% 提升到 20.8%（+18.0%），32B 模型从 27.8% 提升到 34.7%（+6.9%）。最后，在 *GAIA* 上，7B 模型从 7.3% 提升到 9.1%（+1.8%）。

这些收益横跨编码与浏览两类场景，说明统一的跨领域 ADP 训练语料能够在无需领域特定调优的前提下，实现当前最优或接近当前最优的性能，并且在不同模型、动作空间和 agent harness 上都有效。Figure 3 和 Figure 4 也展示出随模型规模增长而来的清晰单调提升，以及 ADP 训练在不同智能体和任务上的稳定增益；在所有规模上，ADP 训练模型都优于对应基础模型。

6.2 多样化数据带来跨任务迁移收益

我们研究**数据多⊠性**是否有助于智能体跨任务泛化。在固定智能体设置与评测方式的情况下，我们比较了三种训练数据混合方式：(i) *Base*（不调优），(ii) 仅做 *Task-specific only* 微调（例如 *SWE-smith Only* 等），以及 (iii) *ADP Data*（详见 § 5），即混合的跨领域语料。如 Table 6 所示，**ADP 在目⊠任务上稳定优于任务特定微调，并且更关键的是，它避免了单领域微调常常在其他任务上引发的负迁移** (Mueller et al., 2024; Kotha et al., 2024; Li et al., 2024)。

具体来看，在 *SWE-Bench* 上，使用 ADP 训练的 `Qwen-2.5-7B-Instruct` 达到 10.4%，而仅用 *SWE-smith Only* 时只有 1.0%；对于 `Qwen-3-8B` (Yang et al., 2025a)，ADP 达到 **16.6%**，相比之下，仅用 *CodeActInstruct + Code-Feedback* 时为 0.2%，仅用 *SWE-smith Only* 时为 11.0%。在 *WebArena* 上，ADP 训练的 `Qwen-2.5-7B-Instruct` 达到 **20.1%**，而 *Go-Browse Only* 为 16.0%。在 *AgentBench OS* 上，ADP 将 `Qwen-3-8B` 提升到 **25.7%**，而 *AgentInstruct Only* 为 21.5%。在 *GAIA* 上，仅用 *AgentInstruct Only* 的准确率只有 0.6%，而 ADP 将其提升到 **9.1%**。总体而言，混合式 ADP 微调比单领域微调带来了更强的域内准确率和跨任务泛化能力。

6.3 ADP 降低了适配新 Agent Harness 的成本

Table 6: 多样化数据与任务特定数据的跨任务迁移对比。对每个基准，我们比较同一 harness+model 在仅任务特定调优和基于 ADP 语料训练下的表现。

智能体	模型	训练数据	准确率
SWE-Bench (Verified) (Jimenez et al., 2024; Chowdhury et al., 2024)			
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-7B-Instruct`	SWE-smith Only	1.0%
	`Qwen-2.5-7B-Instruct`	ADP Data	10.4%
	`Qwen-3-8B`	CodeActInstruct + Code-Feedback	0.2%
	`Qwen-3-8B`	SWE-smith Only	11.0%
	`Qwen-3-8B`	ADP Data	**16.6%**
WebArena (Zhou et al., 2024)			
AgentLab (Drouin et al., 2024) (de Chezelles et al., 2025)	`Qwen-2.5-7B-Instruct`	Go-Browse Only	16.0%
	`Qwen-2.5-7B-Instruct`	ADP Data	**20.1%**
AgentBench OS (Liu et al., 2024b)			
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-3-8B`	AgentInstruct Only	21.5%
	`Qwen-3-8B`	ADP Data	**25.7%**
GAIA (Mialon et al., 2023)			
OpenHands CodeActAgent (Wang et al., 2025)	`Qwen-2.5-7B-Instruct`	AgentInstruct Only	0.6%
	`Qwen-2.5-7B-Instruct`	ADP Data	**9.1%**

Table 7: 将数据集转换为 ADP 所需的 LOC。

数据集	总 LOC
AgentInstruct	∼1500
Code-Feedback	134
CodeActInstruct	269
Go-Browse	335
Mind2Web	476
Nebius SWE-Agent Trajectories	260
NNetNav (live+wa)	290
openhands-feedback	879
Orca AgentInstruct	155
SWE-Gym	221
SWE-smith	228
Synatra	145
Total	**4892**

Table 7 展示了作者和社区贡献者将来自不同来源的 13 个数据集转换到 ADP schema 时所使用的代码行数（LOC）[3]。对于每个数据集，一份 *Raw→ADP* 转换器完成的规范化工作（schema 映射、工具/动作对齐、对话格式整理），本质上与传统针对某个特定 agent harness 的 *Raw→SFT* 转换器所做的工作相同。因此，Table 7 中的 LOC 统计可以合理地作为**没有** ADP 时，每个 agent harness 适配成本的代理指标。

没有 ADP 时。采用这一代理估计，将 D 个数据集转换到 A 个 harness 的成本可近似为 $\text{Cost}_{\text{no-ADP}}(A, D) \approx A \cdot \sum_{i=0}^{D} \text{LOC}_{i,\text{Raw}\to\text{ADP}}$。因此，整个社区的总转换成本是二**次型**的（工作量为 $O(D \times A)$），如 Figure 2 所示。在我们的数据中，13 个数据集的 $\sum_{i=0}^{D} \text{LOC}_{i,\text{Raw}\to\text{ADP}} = 4892$ LOC，因此例如当 $A = 100$ 个 harness 时，总成本约为 $\text{Cost}_{\text{no-ADP}} \approx 100 \times 4892 = \mathbf{489,200}$ LOC。

Table 8: ADP→SFT 转换器所需的 LOC。

Agent Harness	总 LOC
OpenHands CodeActAgent	∼150
SWE-Agent	∼50
AgentLab	∼30
平均	**∼77**

有了 ADP 之后。总成本变为 $\text{Cost}_{\text{ADP}}(A, D) \approx \sum_{i=0}^{D} LOC_{i,\text{Raw}\to\text{ADP}} + \sum_{j=0}^{A} \text{LOC}_{\text{ADP}\to\text{SFT},j}$。因此，如 Figure 2 所示，整个社区的总转换成本变成了**线性**的（工作量为 $O(D + A)$）。Table 8 表明，将 ADP 标准化数据转换为 agent harness 格式平均只需要 77 LOC。对于我们使用的 13 个数据集，当 $A = 100$ 时，$\text{Cost}_{\text{ADP}}(A, D) \approx 4892 + 77 \times 100 = \mathbf{12,592}$，远小于没有 ADP 的情况。此外，新增一个 harness 只需要编写一份将 ADP 标准化数据转换为 SFT 的脚本，这大幅降低了适配新 agent harness 的难度。因此，**ADP 显著降低了社区在开发可扩展、可复现智能体时所需的总体工程投入**。

7 结论与未来工作

ADP 提供了一种实用、轻量的“中间语”，能够将异构数据集统一到一个可被多种 agent harness 消费的单一 schema 中，把今天碎片化的数据格局转变为可扩展的训练流水线。展望未来，我们

[3]所有 LOC 统计均不包括提示文本（例如 system prompt）；只计算转换器代码。

看到三个直接的发展方向。**(i) 多模态：**将 ADP 从文本扩展到图像、屏幕录像以及其他模态，以捕获更丰富的智能体-环境交互。**(ii) 标准化评测：**把同样的标准化“协议”思想应用到评测与环境设置中，使数据集、智能体与评测能够被更自然地组合。**(iii) 社区增长与数据质量：**持续推进开源发布，并引入更强的自动化校验，甚至自动化数据集转换，以在保持质量的同时持续扩大规模。我们相信，通过降低整合成本，并实现跨数据源的系统化、可扩展训练与分析，ADP 能够推动下一波智能体训练研究与实践的发展。

可复现性声明

我们提供了清晰的信息，以支持他人独立复现全部结果。我们描述了 ADP 的模式与转换流水线（§ 3），使其他研究者能够从原始数据源重新生成训练语料。我们在 § 2.1 中列出了所用数据集及其特征。完整的训练与评测设置，包括基础模型、智能体运行框架、我们的 SFT 流水线、评测基准与协议，均在 § 5 中说明。最后，我们开源发布全部代码和数据，包括上述提到的 ADP 模式、转换器与脚本。

致谢

本工作部分得到 Fujitsu 的资助。训练工作得到 CMU FLAME Center 的支持。作者感谢 Professor Daniel Fried 提供的富有洞见的反馈，也感谢 NeuLab 成员以及 LTI ICLR Paper Clinic 的参与者提出的有益建议。

References

All Hands AI. Openhands feedback dataset. https://huggingface.co/datasets/all-hands/openhands-feedback, 2024.

Anthropic Team. The claude 3 model family: Opus, sonnet, haiku. URL https://api.semanticscholar.org/CorpusID:268232499.

Eshta Bhardwaj, Harshit Gujral, Siyi Wu, Ciara Zogheib, Tegan Maharaj, and Christoph Becker. Machine learning data practices through a data curation lens: An evaluation framework. FAccT '24, pp. 1055 – 1067, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704505. doi: 10.1145/3630106.3658955. URL https://doi.org/10.1145/3630106.3658955.

Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-FLAN: Designing data and methods of effective agent tuning for large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 9354–9366, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.557. URL https://aclanthology.org/2024.findings-acl.557/.

Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, et al. Introducing swe-bench verified. https://openai.com/index/introducing-swe-bench-verified, 2024.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=5298fKGmv3. Expert Certification.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 11642–11662. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/drouin24a.html.

Apurva Gandhi and Graham Neubig. Go-browse: Training web agents with structured exploration. *arXiv preprint arXiv:2506.03533*, 2025.

Alexander Golubev, Sergey Polezhaev, Karina Zainullina, Maria Trofimova, Ibragim Badertdinov, Yury Anapolskiy, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergey Skvortsov, Maxim Nekrashevich, Anton Shevtsov, Sergey Abramov, and Boris Yangel. Leveraging training and search for better software engineering agents. *Nebius blog*, 2024. https://nebius.com/blog/posts/training-and-search-for-software-engineering-agents.

Mengkang Hu, Yuhang Zhou, Wendong Fan, Yuzhou Nie, Bowei Xia, Tao Sun, Ziyu Ye, Zhaoxuan Jin, Yingru Li, Qiguang Chen, Zeyu Zhang, Yifeng Wang, Qianshuo Ye, Bernard Ghanem, Ping Luo, and Guohao Li. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation, 2025. URL https://arxiv.org/abs/2505.23885.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European Conference on Computer Vision*, pp. 161–178. Springer, 2024.

Suhas Kotha, Jacob Mitchell Springer, and Aditi Raghunathan. Understanding catastrophic forgetting in language models via implicit inference. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VrHiF2hsrm.

Hongyu Li, Liang Ding, Meng Fang, and Dacheng Tao. Revisiting catastrophic forgetting in large language model tuning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 4297–4308, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.249. URL https://aclanthology.org/2024.findings-emnlp.249/.

Shilong Liu, Hao Cheng, Haotian Liu, Hao Zhang, Feng Li, Tianhe Ren, Xueyan Zou, Jianwei Yang, Hang Su, Jun Zhu, et al. Llava-plus: Learning to use tools for creating multimodal agents. In *European conference on computer vision*, pp. 126–142. Springer, 2024a.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024b. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue. *arXiv preprint arXiv:2402.05930*, 2024.

Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL `http://www.aclweb.org/anthology/P11-1015`.

Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024.

Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.

Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, et al. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024.

Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 6129–6139, 2025.

David Mueller, Mark Dredze, and Nicholas Andrews. Multi-task transfer matters during instruction-tuning. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 14880–14891, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.883. URL `https://aclanthology.org/2024.findings-acl.883/`.

Shikhar Murty, Hao Zhu, Dzmitry Bahdanau, and Christopher D Manning. Nnetnav: Unsupervised learning of browser agents through environment interaction in the wild. *arXiv preprint arXiv:2410.02907*, 2024.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *ICLR*, 2023.

Liangbo Ning, Ziran Liang, Zhuohang Jiang, Haohao Qu, Yujuan Ding, Wenqi Fan, Xiaoyong Wei, Shanru Lin, Hui Liu, Philip S Yu, et al. A survey of webagents: Towards next-generation ai agents for web automation with large foundation models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 6140–6150, 2025.

Tianyue Ou, Frank F. Xu, Aman Madaan, Jiarui Liu, Robert Lo, Abishek Sridhar, Sudipta Sengupta, Dan Roth, Graham Neubig, and Shuyan Zhou. Synatra: Turning indirect knowledge into direct demonstrations for computer agents at scale. In *Conference on Neural Information Processing Systems (NeurIPS)*, Vancouver, BC, December 2024. URL `https://arxiv.org/abs/2409.15637`.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-gym. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=Cq1BNvHx74.

Amandalynne Paullada, Inioluwa Deborah Raji, Emily M Bender, Emily Denton, and Alex Hanna. Data and its (dis) contents: A survey of dataset development and use in machine learning research. *Patterns*, 2(11), 2021.

I Made Putrama and Péter Martinek. Heterogeneous data integration: Challenges and opportunities. *Data in Brief*, 56:110853, 2024. ISSN 2352-3409. doi: https://doi.org/10.1016/j.dib.2024.110853. URL https://www.sciencedirect.com/science/article/pii/S2352340924008175.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383–2392, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264. URL https://aclanthology.org/D16-1264.

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

Yifan Song, Weimin Xiong, Xiutian Zhao, Dawei Zhu, Wenhao Wu, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. AgentBank: Towards generalized LLM agents via fine-tuning on 50000+ interaction trajectories. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 2124–2141, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.116. URL https://aclanthology.org/2024.findings-emnlp.116/.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a. ISSN 2835-8856. URL https://openreview.net/forum?id=ehfRiF0R3a.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024b.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=OJd3ayDDoF.

Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. AgentGym: Evaluating and training large language model-based agents across diverse environments. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and

Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27914–27961, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1355. URL https://aclanthology.org/2025.acl-long.1355/.

Kevin Xu, Yeganeh Kordi, Tanay Nayak, Adi Asija, Yizhong Wang, Kate Sanders, Adam Byerly, Jingyu Zhang, Benjamin Van Durme, and Daniel Khashabi. Tur [k] ingbench: A challenge benchmark for web agents. *arXiv preprint arXiv:2403.11905*, 2024a.

Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials. *arXiv preprint arXiv:2412.09605*, 2024b.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37: 50528–50652, 2024.

John Yang, Kilian Leret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025b.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022b.

Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. Agenttuning: Enabling generalized agent abilities for llms. *arXiv preprint arXiv:2310.12823*, 2023.

Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025.

Jianguo Zhang, Tian Lan, Rithesh Murthy, Zhiwei Liu, Weiran Yao, Ming Zhu, Juntao Tan, Thai Hoang, Zuxin Liu, Liangwei Yang, et al. Agentohana: Design unified data and training pipeline for effective agent learning. *arXiv preprint arXiv:2402.15506*, 2024.

Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Quoc Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Manoj Awalgaonkar, Rithesh R N, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 11583–11597, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.578. URL https://aclanthology.org/2025.naacl-long.578/.

Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 12834–12859, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.762. URL https://aclanthology.org/2024.findings-acl.762/.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024b. Association for Computational Linguistics. URL http://arxiv.org/abs/2403.13372.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oKn9c6ytLx.

A Use of LLMs

We used LLMs to aid and polish writing for style and presentation.

Specifically, LLMs were employed to:

- polish wording, tighten paragraphs, and improve clarity/flow;
- improve latex presentation (e.g., table/figure captions)

B ADP Example Conversion

The conversion pipeline: Raw → ADP → SFT enables scalable training across diverse agent architectures while maintaining data provenance and semantic structure.

This section demonstrates an example conversion from raw agent interaction data of the Code Feedback dataset (Zheng et al., 2024a) to the standardized ADP format. The transformation process extracts structured observations and actions from a raw conversation between the user and the agent.

B.1 Raw Format Example

The raw format typically contains conversational messages with roles and content:

Listing 1: Raw Format Example from Code Feedback

```
[
  {
    "id": 13461,
    "messages": [
      {
        "role": "user",
        "content": "Write a Python script to calculate statistical
            measures..."
      },
      {
        "role": "assistant",
        "content": "You're correct. Let me optimize the code...\n\
           n```python\nimport math\n\ndef calculate_statistics(x,
            y):\n    # Implementation details...\n    return
           r_squared, correlation_coefficient, adjusted_r_squared
           \n```"
      },
      {
        "role": "user",
        "content": "Execution result: \nR-squared: 0.6\
           nCorrelation: 3.87\nAdjusted R-squared: 0.47"
      }
```

```
    ]
  }
]
```

B.2 Standardized ADP Format Example

The standardized format structures the same interaction into typed observations and actions:

Listing 2: Standardized ADP Format Example

```
[
  {
    "id": "13461",
    "content": [
      {
        "class_": "text_observation",
        "content": "Write a Python script to calculate statistical
            measures...",
        "name": null,
        "source": "user"
      },
      {
        "class_": "code_action",
        "language": "python",
        "content": "import math\n\ndef calculate_statistics(x, y):
           \n    # Implementation details...\n    return
           r_squared, correlation_coefficient, adjusted_r_squared
           ",
        "description": "You're correct. Let me optimize the code
           by calculating values once and reusing them..."
      },
      {
        "class_": "text_observation",
        "content": "R-squared: 0.6\nCorrelation: 3.87\nAdjusted R-
           squared: 0.47",
        "name": null,
        "source": "environment"
      },
      {
        "class_": "message_action",
        "content": "<finish> The code executed successfully with
           statistical results...",
        "description": null
      }
    ],
    "details": {}
  }
]
```

The conversion process applies several key transformations:

- **Message Classification**: Raw messages are classified into observations and actions based on content analysis.
- **Code Extraction**: Code blocks within assistant messages are extracted as `code_action` entries.
- **Source Attribution**: User inputs become `text_observation` with `source: "user"`, execution results with `source: "environment"`.

- **Thought Preservation**: Original function thoughts are preserved in `description` fields while structured contents are extracted.
- **Action Classes**: Different classes of agent actions (code execution, messaging, tool usage) are explicitly categorized

This standardization enables systematic analysis of agent behaviors, tool usage patterns, and interaction dynamics across different agent implementations and domains.

B.3 SFT Format Example

The standardized ADP format can be further converted to training-ready formats for specific agent frameworks. Here's the example in OpenHands (Wang et al., 2025) SFT format:

Listing 3: OpenHands SFT Format Example

```
[
  {
    "id": "13461",
    "conversations": [
      {
        "from": "human",
        "value": "Write a Python script to calculate statistical
           measures..."
      },
      {
        "from": "gpt",
        "value": "You're correct. Let me optimize the code...\n\n<
           function=execute_ipython_cell>\n<parameter=code>\
           nimport math\n\ndef calculate_statistics(x, y):\n    #
            Implementation details...\n    return r_squared,
           correlation_coefficient, adjusted_r_squared\n</
           parameter>\n</function>"
      },
      {
        "from": "human",
        "value": "EXECUTION RESULT of [execute_ipython_cell]:\nR-
           squared: 0.6\nCorrelation: 3.87\nAdjusted R-squared: 0
           .47"
      },
      {
        "from": "gpt",
        "value": "<function=finish>\n<parameter=message>\nThe code
            executed successfully with statistical results...\n</
           parameter>\n</function>"
      }
    ],
    "system": "You are OpenHands agent, a helpful AI assistant..."
  }
]
```

C Data Sampling for Balanced Training

To balance domains and reduce over-represented sources, we resample each dataset with a per-dataset multiplier w_d. For dataset d with n_d raw trajectories, we draw $m_d = \lceil w_d \, n_d \rceil$ examples per epoch; if $w_d < 1$ we sample without replacement (downsample), and if $w_d > 1$ we sample with replacement (upsample). This yields an effective mixture proportional to w_d across datasets (and therefore across domains), while keeping the overall epoch size stable.

Table 9: Per-dataset sampling multipliers w_d. $w_d < 1$ indicates downsampling; $w_d > 1$ indicates upsampling.

Dataset	w_d	Direction
`agenttuning_alfworld`	2	up
`agenttuning_db`	2	up
`agenttuning_kg`	2	up
`agenttuning_mind2web`	2	up
`agenttuning_os`	2	up
`agenttuning_webshop`	2	up
`code_feedback`	0.1	down
`codeactinstruct`	1	neutral
`go-browse-wa`	1	neutral
`mind2web`	1	neutral
`nebius_SWE-agent-trajectories`	0.2	down
`nnetnav-live`	1	neutral
`nnetnav-wa`	1	neutral
`openhands`	1	neutral
`orca_agentinstruct`	0.001	down
`swe-gym_openhands_sampled_trajectories`	3	up
`swe-smith`	1	neutral
`synatra`	0.01	down

In practice, we fix a random seed for reproducibility and shuffle the union of sampled examples across datasets each epoch. This scheme targets a more balanced distribution across coding, SWE, tool-use, and web-browsing sources by attenuating very large corpora (e.g., `orca_agentinstruct` at w_d=0.001) and amplifying under-represented ones (e.g., `swe-gym_openhands_sampled_trajectories` at w_d=3).

C.1 Domain-Specific Data Filtering

Beyond balanced sampling, we apply domain-specific filtering to optimize training effectiveness for each agent framework based on their evaluation focus and capabilities.

OpenHands and SWE-Agent Training Data. For OpenHands CodeActAgent and SWE-Agent, which are primarily evaluated on coding and software engineering tasks (SWE-Bench, AgentBench OS, and GAIA), we use only the *non-web* portion of the ADP training corpus. This includes datasets focused on code generation, software engineering, general agent instruction following, and API/tool usage. Specifically, we exclude web browsing datasets Mind2Web, Go-Browse, NNetNav, and Synatra to avoid potential interference from web-specific interaction patterns that are not applicable to command-line and coding environments. Thus, using the sampling multipliers in Table 9, the total number of training samples used is around 30K. Future experiments could explore different sampling multipliers and examine the effect of each dataset on coding and software engineering tasks.

AgentLab Training Data. For AgentLab, which is designed for web browsing tasks and we evaluated exclusively it on WebArena, we use only the *web* portion of the ADP training corpus. This includes datasets focused on web navigation, browser-based task completion, and web-specific agent instruction following (Mind2Web, Go-Browse, NNetNav, and Synatra). We exclude coding and software engineering datasets to ensure the model is optimized for web browsing patterns and UI element interaction without dilution from less compatible domains. Thus, using the sampling multipliers in Table 9, the total number of training samples used is around 20K. Future experiments could explore different sampling multipliers and examine the effect of each dataset on web tasks.

D Performance Scaling

Figure 3 and Figure 4 shows the scaling curve of performance and performance gains across agents and benchmarks. Both plots show clear monotonic gains regardless of model size

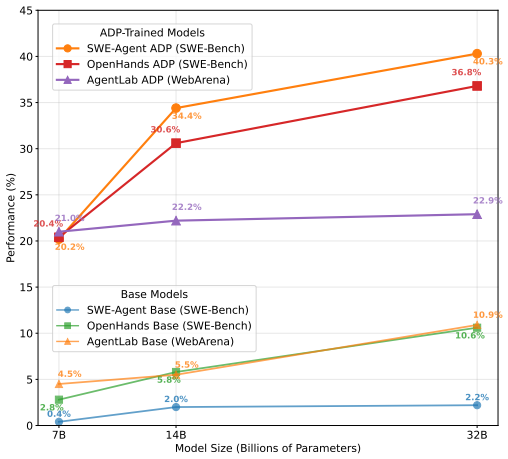

Figure 3: Performance Scaling Across Agents and Benchmarks (Base vs ADP Trained)

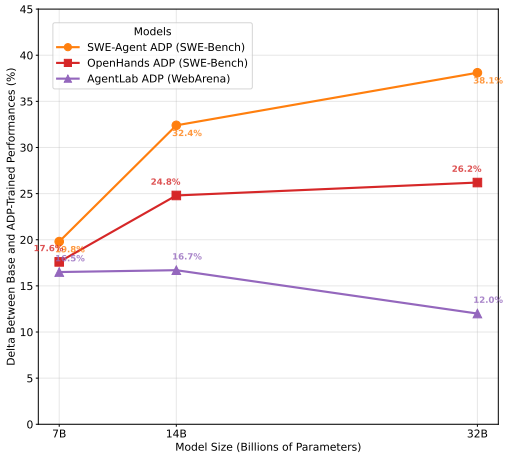

Figure 4: Performance Gains Across Agents and Benchmarks.

and consistent boosts from ADP training across agents and tasks, with ADP-trained models outperforming their base counterparts at every scale.

E Additional Experiments

E.1 ADP's Advantage Persist Under Equal Data Scale

To address the question of fair data scaling, we additionally compare ADP against a single-domain fine-tuning baseline under matched dataset size. Specifically, we train `Qwen-3-8B` on SWE-smith with up-sampling to match the number of training examples used in the ADP mixture, and evaluate both models on SWE-Bench with the OpenHands harness. As shown in Table 10, SWE-smith training yields 11.0% accuracy, whereas ADP training achieves 16.6% under a comparable number of samples. This demonstrates that ADP's benefit does not stem from data volume alone, but from the greater diversity and unified structure of the ADP corpus.

Table 10: Equal-scale comparison of `Qwen-3-8B` trained on SWE-smith vs. ADP, evaluated on SWE-Bench with the OpenHands harness.

Model	Training Data	Data Scale	Accuracy
`Qwen3-8B`	SWE-smith (up-sampled)	$\approx$ 30K	11.0%
`Qwen-3-8B`	ADP	$\approx$ 30K	**16.6%**

F License of Use

This section provides licensing information for all datasets referenced in Table 1 and used in our experiments. We have made every effort to identify and respect the licensing terms of each dataset. Users should verify current licensing terms before using these datasets. Users should also verify the licensing terms of datasets they are adding to ADP.

F.1 Dataset Licenses

License Compliance: We have ensured compliance with licenses of all datasets utilized in this paper. All licenses permit research use.

F.2 Usage Guidelines

When using the ADP-converted versions of these datasets:

Table 11: Licensing information for datasets used in ADP

Dataset	License	Link
AgentInstruct	Apache 2.0	ZhipuAI/AgentInstruct
Code-Feedback	Apache 2.0	m-a-p/Code-Feedback
CodeActInstruct	Apache 2.0	xingyaoww/code-act
Go-Browse	MIT	ApGa/Go-Browse
Mind2Web	CC BY 4.0	osunlp/Mind2Web
Nebius SWE Trajectories	CC BY 4.0	nebius/SWE-agent-trajectories
NNetNav-live	Apache 2.0	stanfordnlp/nnetnav-live
NNetNav-wa	Apache 2.0	stanfordnlp/nnetnav-wa
openhands-feedback	MIT	all-hands/openhands-feedback
Orca Agentinstruct	CDLA-Permissive-2.0	microsoft/orca-agentinstruct-1M-v1
SWE-Gym	MIT	SWE-Gym/SWE-Gym
SWE-smith	MIT	SWE-bench/SWE-smith-trajectories
Synatra	CC BY-SA 4.0	oottyy/Synatra

1. **Verify Current Licenses**: Check the original dataset repositories for the most up-to-date licensing terms
2. **Respect Restrictions**: Some datasets have restrictions on commercial use, redistribution, or specific use cases.
3. **Cite Appropriately**: Include citations for both the original datasets and the ADP conversion methodology.
4. **Contact Authors**: For datasets with unclear licensing, contact the original authors for clarification on usage terms.

F.3 Disclaimer

Licenses were collected at the time of dataset integration and may have changed. Users are responsible for verifying current licensing terms and ensuring compliance with all applicable licenses. The ADP project does not assume responsibility for license violations by downstream users.

For questions about specific dataset licenses or usage permissions, please contact the original dataset authors or maintainers directly.